

GOOGLE CLOUD STORAGE COMPATIBILITY

Connect Your Google Cloud Storage Workloads to Inaya

A step-by-step guide for pilot IT/ops teams: use Inaya's real Google Cloud Storage-compatible XML API endpoint with your existing HMAC credentials, tools, and client libraries.

Pilot Guide - Google Cloud Storage Compatibility - September 2026

SECTION 01

What This Gives You

Inaya's storage endpoint accepts Google Cloud Storage's own documented XML API authentication conventions -- both GCS's native GOOG4-HMAC-SHA256 signing scheme and its AWS4-HMAC-SHA256 interoperability mode. If your existing code, scripts, or client libraries authenticate to Google Cloud Storage using an HMAC access key and secret, they can point at Inaya instead by changing only the endpoint and credential -- no protocol rewrite. Every byte is still encrypted, sharded, and redundantly stored across Inaya's decentralized storage network underneath.

ENDPOINT	ACCEPTED SIGNING MODES
https://<your-inaya-host>/api/s3	GOOG4-HMAC-SHA256 (Google's native scheme) and AWS4-HMAC-SHA256 (Google's own documented S3-interoperability mode)

What to put in place of <your-inaya-host>. Same as the Multi-Cloud Storage Compatibility guide: for the standard shared Inaya platform, use www.inayanetwork.com. If your pilot runs on a dedicated hostname, your Inaya account manager will provide it.

SECTION 02

Step 1 - Get Your Credentials

1. **Sign in to Business Workspace.** Open the "S3-Compatible Storage" panel from the left navigation -- the same panel used for AWS and Azure access.
2. **Click "+ New S3 credential."** One credential works across AWS, Azure, and Google Cloud Storage signing conventions -- you do not need a separate credential per cloud.
3. **(Optional) Restrict its scope.** Limit the credential to one bucket, one folder path, specific operations (READ/WRITE/DELETE/LIST), and/or an expiry date before creating it.
4. **Save the Access Key ID and Secret Access Key immediately.** Shown exactly once. Store it the way you would store any other cloud credential.

SECTION 03

Step 2 - Authenticate with Google's Native GOOG4 Scheme

If your existing code uses Google's official client libraries (for example, the Node.js `@google-cloud/storage` package or the Python `google-cloud-storage` package) configured with an HMAC key pair and a custom API endpoint, it produces a native GOOG4-HMAC-SHA256 signature. Point it at Inaya by overriding the endpoint and supplying your Inaya-issued HMAC credential in place of a Google one.

CONCEPTUAL EXAMPLE (CONSULT YOUR SPECIFIC CLIENT LIBRARY'S OWN DOCUMENTATION FOR THE EXACT ENDPOINT-OVERRIDE OPTION NAME)

```
endpoint: "https://<your-inaya-host>/api/s3"  
hmacAccessId: "<your Access Key ID>"  
hmacSecret: "<your Secret Access Key>"
```

Inaya's server verifies the real, published GOOG4-HMAC-SHA256 algorithm exactly as Google documents it (canonical request, x-goog-date/x-goog-content-sha256 headers, the goog4_request credential scope) -- this is not a simplified approximation.

SECTION 04

Step 3 - Authenticate with the AWS4 Interoperability Mode

Google also documents an "S3 interoperability" signing mode for its own XML API -- genuine AWS4-HMAC-SHA256, byte-identical to real AWS SigV4. Any tool built to speak this mode (including the standard AWS CLI and AWS SDKs, pointed at a custom endpoint) already works against Inaya with zero changes.

TERMINAL -- USING THE AWS CLI

```
aws configure --profile inaya
# AWS Access Key ID:     <your Access Key ID>
# AWS Secret Access Key: <your Secret Access Key>
# Default region name:   inaya

aws s3 ls --profile inaya --endpoint-url https://<your-inaya-host>/api/s3
aws s3 cp report.pdf s3://pilot-bucket/ --profile inaya --endpoint-url https://<your-inaya-host>/api/s3
```

SECTION 05

Step 4 - Verify the Pilot Connection

1. **1. Create a test bucket.** Using whichever authentication path from Steps 2-3 matches your existing tooling.
2. **2. Upload a small test file, then list the bucket to confirm it appears.**
3. **3. Download it back and compare.** The downloaded file must be byte-for-byte identical to what you uploaded.
4. **4. Confirm it's visible in Business Workspace.** Confirms Inaya's real storage/encryption/redundancy pipeline received it, not just a local acknowledgment.
5. **5. Delete the test object and bucket.** Confirms delete permissions work end-to-end before moving real data.

SECTION 06

What's Tested, and One Honest Limitation

- Object create/read/list/delete, byte-range downloads, and metadata: tested and working via both signing modes above.
- Large-file uploads via Google's XML API multipart mechanism (the same shape as AWS S3 multipart): tested and working.

gsutil's S3-compatible mode -- currently untested, and here's exactly why. While validating this feature, we found a real bug in Google's own gsutil tool: its S3-compatibility code crashes when pointed at any address that doesn't look like a standard Amazon web address (an issue in Google's own bundled software, confirmed by tracing the exact failing line -- unrelated to Inaya). Until Google fixes this upstream, we can't responsibly claim gsutil's S3-compatible mode is tested against Inaya, even though the underlying signing method it uses is the same AWS4-HMAC-SHA256 mode already proven to work above. If your workflow depends specifically on gsutil, talk to your Inaya technical contact before relying on it for production use.

Not yet supported this pass: signing in with a Google account/OAuth directly, temporary signed download links, and addressing a bucket via a subdomain (bucket.storage.googleapis.com style) rather than a path. None were required by a validated pilot workload; ask your Inaya contact if your workflow needs one.

SECTION 07

Getting Help

During the pilot, route connection issues or questions about scope/permissions to your assigned Inaya technical contact rather than general support -- pilot feedback shapes the general-availability release.